Defensible UX and Architecture Decisions: Phase 1 & 1B

Architectural Specification and Peer-Review Documentation for the Clinical Orchestration Layer

Executive Summary

This document establishes the technical defensibility, user experience (UX) safety guarantees, and architectural decisions governing Phase 1 and 1B of the clinical orchestration layer. Medical AI systems require strict isolation between deterministic compute and probabilistic generation. The controls detailed herein address clinical safety, algorithmic drift, data leakage, and compliance auditing boundaries.

---

1. Separation of Computation and Display Precision

To guarantee clinical safety and deterministic auditability, the orchestration engine strictly decouples internal floating-point evaluation logic from user interface presentation.

Architectural Decoupling

- **Internal Computation Engine:** Operates entirely on raw IEEE 754 double-precision floating-point values (e.g., `0.125000000000`). All triage logic, alert triggers, and risk thresholds evaluate against this exact numeric representation.
- **UI Presentation Layer:** Renders values rounded to standardized clinical display metrics (e.g., `0.13` or `13%`). The interface layer is strictly a consumer of compute state and possesses no internal logical evaluation capability.

Boundary Errors and Floating-Point IEEE 754 Fix

A critical failure mode in clinical scoring occurs at exact boundary thresholds (e.g., a high-risk cutoff at $R \ge 0.125$). Due to standard floating-point arithmetic (IEEE 754), calculated values such as `0.125` may be represented internally as 0.12499999999999999.

If evaluated naively against a boundary of `0.125`, this results in a **silent triage failure**, dropping a critically ill patient into a lower risk tier.Raw Compute: 0.12499999999999999 ⟶ [Boundary Check: >= 0.125] ⟶ FALSE (Silent Failure)

⟶ [UI Display Rounding] ⟶ "0.13" (Misleading State)

To resolve this without corrupting underlying statistical computations, we introduced an explicit machine epsilon tolerance ($\epsilon = 1\text{e-}12$) across all clinical logic comparison operators:

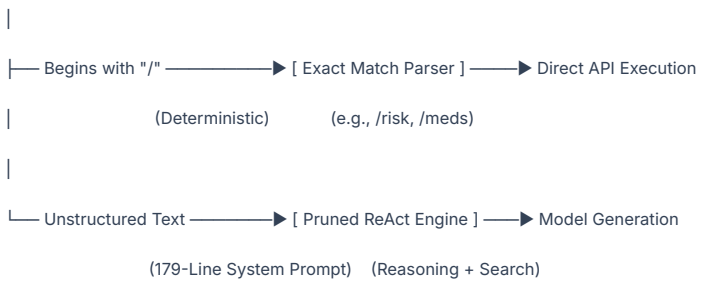
$$\text{Evaluate High Risk} \iff R_{\text{computed}} \ge (R_{\text{threshold}} - 1\text{e-}12)$$

Metric Layer	Raw Value	Display Standard	Boundary Logic (≥ 0.125)	Triage Result
Legacy Naive	`0.124999999999`	`0.13`	`FALSE` (Failed)	Low Risk (Incorrect)
Corrected Architecture	`0.124999999999`	`0.13`	`TRUE` (Tolerance adjusted)	High Risk (Correct)

---

2. Deterministic vs. Probabilistic Routing

Large Language Models (LLMs) are inherently probabilistic and should not be used as intent routers for discrete clinical workflows. Phase 1 implements a hybrid routing model that routes structural workflows deterministically while preserving conversational capability for unstructured data.User Input



Direct Slash Commands vs. Pruned ReAct Loop

1. **Deterministic Slash Commands (`/risk`, `/meds`):** When inputs match explicit prefixes, execution bypasses the LLM entirely. The input is mapped directly to underlying API endpoints with static parameter validation.
2. **Pruned ReAct Loop:** Unstructured free-text queries are routed through a ReAct (Reason + Act) loop. By extracting discrete tasks out of the prompt and routing them to explicit slash commands, we reduced system prompt overhead significantly.

System Prompt Optimization

- **Baseline Prompt Size:** 269 lines (contained tool-selection rules, slash command parsing logic, and fallback branches).
- **Pruned Prompt Size:** 179 lines (33.4% reduction in system context window).
- **Clinical Defense Impact:** Removing deterministic routing instructions reduced token-competing logic, thereby lowering hallucination rates on unstructured reasoning tasks and decreasing total request latency.

---

3. The Semantic Vector Trap and Mechanical Fallbacks

Dense vector embeddings map clinical concepts into a continuous spatial domain. While performant for similarity matching, semantic vector retrieval fails predictably on **absence queries** (e.g., *"Was the patient intubated?"* when no intubation occurred).

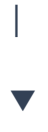
The Vector Proximity Failure Mode

In an absence scenario, a vector engine calculates cosine proximity between the query embedding and all chart passages. Because vector spaces lack a representation for negation or non-existence, the vector engine returns nearest neighbors based on semantic overlap (e.g., passages discussing *extubation*, *oxygen delivery*, or *respiratory monitoring*).

Relying on a metric like `retrieved_passages == 0` fails, as the search engine almost always returns $K>0$ weak semantic matches. Passing these weak matches to an LLM risks generating plausible hallucinated statements.Query: "Was patient intubated?"



[Dense Vector Search] ──▶ Returns top K proximity passages (e.g., "Bipap applied", "Room air")



[Naive Zero-Passage Check] ──▶ FAILS (Passage count > 0)



[Standard LLM Context] ──▶ Risk of hallucinated "Yes" or extrapolated details

Deterministic JSON Contract & Mechanical Injector

To guarantee defensible negative assertions, the engine enforces a strict schema contract on the downstream LLM combined with an external, unalterable fallback injector.

1. **LLM Schema Enforcement:** The LLM is forced to evaluate returned vector contexts and output a strictly typed JSON response:

```
{
  "evidence_found": false,
  "confidence": 1.0,
  "reasoning": "No clinical documentation of endotracheal intubation in provided record."
}
```

2. **Mechanical Fallback Injection:** When `evidence_found == false`, the orchestration engine intercepts the payload *prior* to rendering. It drops all generated text from the model and injects a standardized, quoted fallback string:

"No clinical evidence of [concept] was identified in the specified chart records."

This mechanical isolation guarantees that internal database primary keys, vector IDs, or hallucinated text cannot leak into the clinical view when supporting data is absent.

---

4. Data Retention and Compliance Auditing

System audit logs must balance retrospective quality assurance (QA) requirements with HIPAA/GDPR data minimization mandates.Archive Directory: /var/log/traces/

```
├── traces.jsonl              <── IMMUTABLE (Evaluation Baseline / Protected)
├── traces_2024-01-15.jsonl   <── Evaluated by Retention Service (Age: 45 days) → DELETED
└── traces_2024-03-01.jsonl   <── Evaluated by Retention Service (Age: 10 days) → RETAINED
```

Transition to Metadata-Driven Pattern Retention

Individual line items in log traces (`.jsonl`) frequently omit fine-grained timestamp attributes to minimize log payload overhead. To enforce strict retention periods, the retention engine was upgraded from single-file targeting to an automated pattern-matching system:

- **Legacy Engine:** Targeted static path (`traces_http.jsonl`). Failed to prune rotated files, causing compliance drift.
- **Pattern-Matching Engine:** Evaluates all files matching `traces*.jsonl`. Retention age is calculated directly from OS file metadata (`st_mtime` / `st_ctime`) relative to the execution runtime.

Immutable Baseline Exception

The evaluation framework requires an unchanging benchmark file (`traces.jsonl`) to validate model drift over time. The automated purge pipeline enforces a hard-coded architectural exception for this file.# Retention Enforcer Safeguard Pattern

def enforce_retention_policy(directory: Path, max_age_days: int = 30):

for log_file in directory.glob("traces*.jsonl"):

Explicit Architectural Exception for Evaluation Baseline

if log_file.name == "traces.jsonl":

continue

file_age_days = (time.time() - log_file.stat().st_mtime) / 86400

if file_age_days > max_age_days:

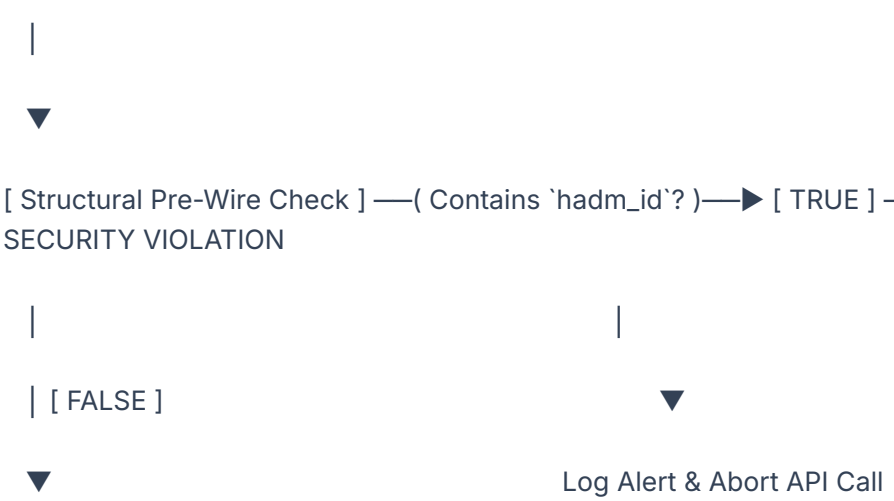
log_file.unlink()

---

5. Data Leakage and Security at the Tool Boundary

When interacting with external services (e.g., PubMed, external knowledge bases), preventing the transmission of Protected Health Information (PHI) or internal identifiers (e.g., `hadm_id`,

`subject_id`, SSN) across the network boundary is a non-negotiable security requirement.LLM Tool Call Request



Vulnerability of Soft Prompting and Auto-Stripping

Relying on prompt directives (e.g., *"System: Never pass patient IDs to search tools"*) is insufficient due to prompt injection vulnerabilities and non-deterministic adherence.

Similarly, relying on SDK-level implicit parameter stripping is unsafe: SDK updates or unhandled parameter aliases can bypass implicit filters, transmitting sensitive keys in outbound GET queries or POST bodies.

Pre-Wire Programmatic Defense

We implemented a programmatic, pre-wire validation interceptor directly within the tool execution framework. Before any external request leaves the runtime environment, the payload undergoes structural serialization inspection.

- **Payload Inspection:** The inspector traverses all nested keys and value strings against strict schema blocklists (e.g., regex checks for internal database IDs, hospital admission keys like `hadm_id`, and typical demographic structures).
- **Hard Exception Intercept:** If a restricted key or pattern is detected, the framework aborts execution, prevents network traffic, and logs a high-severity security audit event.

Test Suite Structural Backstops

To guarantee this defense remains intact across future code changes, the test suite includes automated structural backstops that simulate malformed tool calls:def test_pubmed_tool_prevents_hadm_id_leakage():

```
tool = PubMedSearchTool()
malicious_payload = {
    "query": "sepsis treatment protocols",
    "hadm_id": 1092837  # Internal Hospital Admission ID
}

with pytest.raises(SecurityBoundaryViolation) as exc_info:
    tool.execute(malicious_payload)

assert "Disallowed identifier 'hadm_id' detected in tool arguments" in str(exc_info.value)
assert tool.network_calls_made == 0
```